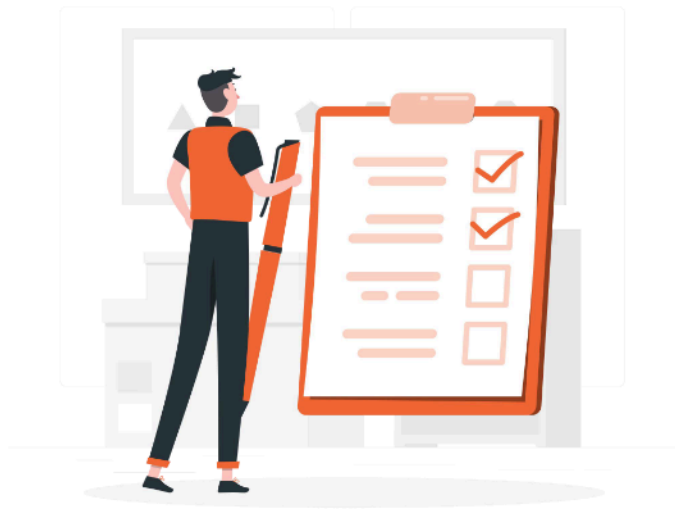




LLM Evaluation Metrics, Safety and Toxicity Detection

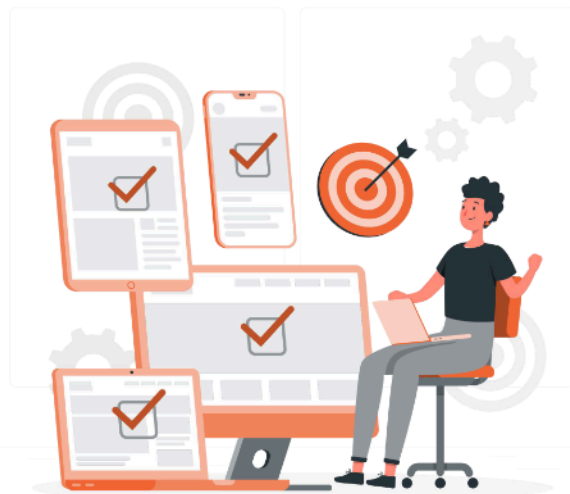
Lesson

1. BLEU, ROUGE, METEOR
2. Embedding similarity
3. Groundedness & factuality scoring
4. Azure OpenAI Safety System
5. Analyzing toxic output
6. Bias mitigation techniques



Learning Outcome

1. Understand classical and modern LLM evaluation metrics
2. Differentiate lexical, semantic, and factuality-based evaluation
3. Apply Azure OpenAI safety capabilities in production
4. Detect, analyze, and mitigate toxicity and bias
5. Design an end-to-end evaluation pipeline



BLEU, ROUGE, METEOR



Why We Need Metrics for LLMs

THE PROBLEM

"LLMs produce different output every time you run them. Traditional unit tests cannot grade fluent writing — we need automatic metrics that compare the model's output to a known good answer."

In plain English:

Think of grading 10,000 student essays. You can't read them all yourself — you need a fair, automatic scoring system.

What "good" looks like



Correct meaning

The output says the right thing.



Faithful to source

Nothing is invented or distorted.



Safe and respectful

No toxic, biased, or harmful language.



Fast to compute

Runs in CI on every change.

The Recurring Grading Analogy

SETUP

"A teacher gives students a translation task. The teacher already has a perfect "model answer". To grade hundreds of student answers fairly, the teacher invents an automatic scoring rule."

In plain English:

BLEU, ROUGE, and METEOR are three different "grading rules" — they each ask a slightly different question about the student's answer.

Three different graders



BLEU — the word-matcher

"Did you use the same words I used?"



ROUGE — the coverage checker

"Did you mention all my key points?"



METEOR — the smart matcher

"Same words OR synonyms count too."

BLEU: The Word-Matcher

WHAT IS BLEU?

"BLEU (Bilingual Evaluation Understudy) checks how many small word-groups in the candidate also appear in the reference. The more matches, the higher the score."

— Papineni et al., 2002

In plain English:

"How much of what you wrote also appears in my model answer?" Score is between 0 (nothing matches) and 1 (perfect match).

How BLEU thinks



Looks at small chunks

1-word, 2-word, 3-word, 4-word groups (n-grams).



Counts overlap

How many of those chunks appear in the reference?



Penalizes "too short"

If you wrote one word, you can't game the score.



Most-cited NLP metric

Originally for machine translation in 2002.

BLEU: How the Score is Computed

- Compute precision for each n-gram size (1, 2, 3, 4)
- Take the geometric mean of the four precisions
- Multiply by Brevity Penalty (BP) so short answers can't cheat
- BP = 1 if candidate is long enough; less than 1 if too short
- Final score is between 0 and 1 — typically reported as 0–100

WORKED EXAMPLE

Reference:
"the cat sat on the mat"

Candidate:
"the cat sits on mat"

1-gram matches: 4 / 5 = 0.80

2-gram matches: 2 / 4 = 0.50

BP ≈ 0.82 (a bit short)

BLEU ≈ 0.45

FORMULA

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right)$$

$w_n = \frac{1}{4}$ for $n = 1, 2, 3, 4$

BLEU: Strengths and Weaknesses

✓ Strengths

- Fast and deterministic — same input gives the same score
- Cheap to run in CI on every model change
- Great for translation tasks with one clear "right" answer
- Easy to explain and to track over time as a baseline
- Widely understood — appears in nearly every NLP paper

✗ Weaknesses

- Treats "car" and "automobile" as completely different words
- Penalizes valid paraphrases that mean the same thing
- Sensitive to tokenization — different tokenizers = different scores
- Weak correlation with human judgment for chat and creative tasks
- Often misleading for open-ended LLM responses

ROUGE: The Coverage Checker

WHAT IS ROUGE?

"ROUGE (Recall-Oriented Understudy for Gisting Evaluation) flips the BLEU question: instead of asking "what fraction of your words are right?", it asks "what fraction of the important content did you cover?""

— Chin-Yew Lin, 2004

In plain English:

"Did your summary mention everything that mattered in the reference?" Designed for summarization tasks.

Why "recall" matters



Recall over precision

BLEU asks "how much of yours is right?" — ROUGE asks "how much of mine did you capture?"



Built for summaries

Originally designed to evaluate news article summaries.



Several variants

ROUGE-N, ROUGE-L, ROUGE-W, ROUGE-S — each measures coverage differently.



Standard for RAG

Often used today to measure how well RAG answers cover retrieved documents.

ROUGE: The Four Common Variants

Variant	What it measures	Best for	Plain-English question
ROUGE-N	N-gram recall (typically $N = 1$ or 2)	Headlines, short summaries	"How many of my key words did you use?"
ROUGE-L	Longest Common Subsequence — captures sentence-level structure	Paragraph summaries, RAG answers	"Did the order of your ideas roughly match mine?"
ROUGE-W	Weighted LCS — favors consecutive matches over scattered ones	Fluent prose summaries	"Did you keep the phrases together?"
ROUGE-S	Skip-bigram co-occurrence with arbitrary gaps	Flexible word-pair relationships	"Did you connect the same concept pairs as I did?"

METEOR: The Smart Matcher

WHAT IS METEOR?

"METEOR fixes BLEU's biggest blind spot: it understands that "buy" and "purchase" mean roughly the same thing. It uses stemming, synonyms (from WordNet), and paraphrase tables."

— Banerjee & Lavie, 2005

In plain English:

"BLEU with a thesaurus." Same idea as BLEU, but it gives credit when you use synonyms or different word forms.

What METEOR adds



Stemming

"running" matches "run" — same root counts.



Synonyms

"big" matches "large" — uses WordNet.



Paraphrase tables

"kick the bucket" matches "die" in some versions.



Word-order penalty

Penalizes scrambled word order, even with right words.

Side-by-side: Same answer, 3 scores

SCENARIO

A user asked the LLM to translate one Indonesian sentence to English. We will grade the same response with all three metrics.

REFERENCE (the "model answer")

The cat is sleeping on the carpet.

CANDIDATE (what the LLM produced)

A feline is napping on the rug.

BLEU

0.00

Almost no exact word overlap. BLEU sees this as a totally wrong answer.

ROUGE

0.20

Some structural overlap, but most words don't match. Low recall.

METEOR

0.61

Recognizes "cat $\approx$ feline", "sleeping $\approx$ napping", "carpet $\approx$ rug" via synonyms.

When to Use Which Metric

Use case	Best metric	Why	Watch out for
Machine translation, code generation	BLEU	Output is short with one clear "right" answer	Misleading on free-form chat
Summarization, RAG answers	ROUGE	Recall matters — did the summary cover everything?	Encourages copying long passages
Translation when paraphrases matter	METEOR	Synonyms and stems give credit for valid paraphrasing	Slower; needs language-specific resources
Conversational chat, creative writing	None of the above	All three correlate poorly with human judgment here	Use embedding similarity or LLM-as-Judge instead

Bottom line: lexical metrics are great for fast regression tests, but they are not enough on their own for modern LLM products.

Embedding Similarity



Why Words Are Not Enough

THE PROBLEM

"I want to cancel my credit card" and "Please terminate my card" mean the same thing — but BLEU, ROUGE, and METEOR all give them a near-zero score because the words look different."

In plain English:

For real chat assistants, we need to compare meaning, not strings. That is exactly what embeddings let us do.

What goes wrong



Paraphrase blindness

Same meaning, different words → bad score.



Translation variation

Two correct translations can look totally different.



Conversational style

Casual vs formal phrasing — same answer, low BLEU.



No "rough match"

"Almost right" gets the same score as "totally wrong".

What are Embeddings

WHAT IS AN EMBEDDING?

"An embedding is a list of numbers (e.g. 768 numbers) that represents the meaning of a piece of text. Texts with similar meaning end up with similar number lists."

In plain English:

Think of it as GPS coordinates for meaning. Two pieces of text close together on the "meaning map" mean similar things.

Properties of embeddings



Coordinates for meaning

Each text becomes a point in high-dimensional space.



Distance = similarity

Closer points → more similar meaning.



Learned from huge corpora

Models like BERT, GPT, BGE produce embeddings.



Foundation for RAG

Same trick that powers semantic search.

Cosine Similarity: The Angle Between Meanings

- Treat each text as a vector (a list of numbers — the embedding)
- Cosine similarity measures the angle between two vectors
- Output ranges from -1 (opposite) to +1 (identical)
- For text embeddings, scores are typically between 0 and 1
- Magnitude does not matter — only direction (so sentence length is OK)
- A score above ~0.85 typically means "very similar meaning"

WORKED EXAMPLE

"cancel my card"

→ A = [0.6, 0.8, 0.0]

"terminate my account"

→ B = [0.7, 0.5, 0.1]

A · B = 0.82

||A|| = 1.0 ||B|| ≈ 0.866

cos θ ≈ 0.95

→ Very similar meaning ✓

FORMULA

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

Other Similarity Measures

Measure	What it captures	Use when...	Caveat
Cosine	Direction (angle between vectors)	Default for normalized text embeddings	Ignores magnitude
Dot product	Direction $\times$ magnitude (= cosine if normalized)	Models trained with dot-product loss	Sensitive to vector norms
Euclidean	Straight-line distance between vectors	When magnitude carries meaning (counts)	In high dimensions, distances flatten out
Manhattan	Sum of absolute differences (L1)	Sparse vectors, robustness to outliers	Less common in modern embedding stacks

BERTScore: Token-Level Meaning Matching

WHAT IS BERTSCORE?

"BERTScore uses BERT embeddings to match every token in the candidate to the most similar token in the reference. Then it computes precision, recall, and F1 from those matches."

— Zhang et al., ICLR 2020

In plain English:

Like BLEU, but each word "vote" is partial — close synonyms get partial credit instead of zero.

Properties of embeddings



Coordinates for meaning

Each text becomes a point in high-dimensional space.



Distance = similarity

Closer points → more similar meaning.



Learned from huge corpora

Models like BERT, GPT, BGE produce embeddings.



Foundation for RAG

Same trick that powers semantic search.

Sentence Embedding Models You Should Know



Sentence-BERT (SBERT)

Reimers & Gurevych, 2019. The original siamese-BERT for fast sentence vectors.



OpenAI text-embedding-3

Strong general-purpose embeddings; available via Azure OpenAI.



BGE-M3 (multilingual)

Open-source, supports 100+ languages including Bahasa Indonesia.



E5 family

Microsoft's open-source multilingual embeddings; viable on-prem option.



STS Benchmark

The standard benchmark to compare encoders on semantic similarity.



Use case

Compare LLM answer to gold answer; rank retrieved documents in RAG.

Practical Considerations for Production

Cost

API calls add up. Self-hosted GPU shifts cost from per-call to capex.

Latency

Online evaluation adds milliseconds — batch and cache aggressively.

Embedding drift

Changing the encoder version invalidates stored vectors and shifts thresholds.

Threshold calibration

What does cosine = 0.85 mean? Tune per task with a labeled dev set.

Similarity ≠ correctness

Two confidently-wrong answers can look very similar to each other.

Language coverage

Verify your encoder handles Bahasa Indonesia, Javanese, code-switching.

Groundedness & Factuality Scoring



The Hallucination Problem



What is hallucination?

Plausible-sounding output that is not actually supported by any source.



Intrinsic

Contradicts the provided source — model invents detail not in the docs.



Extrinsic

Cannot be verified from the source — may or may not be true elsewhere.



Enterprise impact

Wrong product specs, fabricated policies, invented features — direct customer harm.



RAG mitigates, not eliminates

Retrieval helps a lot, but does not stop the model from going off-context.



Why detection matters

You cannot fix what you cannot measure. Every production LLM needs hallucination monitoring.

Groundedness vs Factuality: Related but Different



Groundedness

- Is the answer supported by the provided context (retrieved docs)?
- Closed-world: only the supplied source counts as evidence
- Directly testable in RAG via context-vs-answer comparison
- Methods: NLI entailment, citation matching, attribution scoring
- A grounded answer can still be wrong if the source itself is wrong



Factuality

- Is the answer true in the real world, regardless of the input context?
- Open-world: requires external knowledge or trusted reference
- Methods: knowledge-base lookup, FactScore, TruthfulQA
- Requires gold sources — costly to construct and maintain
- A factual answer can still be ungrounded if it does not use the source

Reference-based Factuality Methods



FEVER pipeline

Claim → retrieve evidence → verdict (Supports / Refutes / Not Enough Info).



FactScore

Decompose answers into atomic facts, then verify each one against a source.



TruthfulQA

Benchmark of questions designed to elicit common false beliefs.



KB verification

Match named entities and relations against a curated knowledge graph.



Strength

High precision when a claim disagrees with a trusted source.



Weakness

Coverage gaps — if the KB does not contain the topic, no verdict is possible.

Reference-free Methods: When No Gold Answer Exists



Self-consistency

Sample multiple completions; check whether the model agrees with itself.



SelfCheckGPT

Compare a primary answer against several stochastic re-samples.



Chain-of-Verification

Model drafts an answer, generates verification questions, re-checks each claim.



Why it works

Hallucinations are inconsistent across samples; well-grounded facts stay stable.



Implementation

Cheap to add to any LLM pipeline — only requires extra inference calls.



Caveat

Confidently-wrong models can be consistently wrong — these methods reduce, not eliminate, risk.

RAG-specific Groundedness Evaluation



Did the model use the context?

Was the retrieved passage actually used, or did the model fall back to memory?



Token-level attribution

Identify which answer tokens trace back to which spans of which document.



NLI-based scoring

Use a Natural Language Inference model to test if context entails each claim.



Citation accuracy

When the model cites sources, do those sources actually support the claim?



Coverage vs precision

Stricter grounding reduces helpfulness; looser grounding increases risk.



Production application

Critical for any internal RAG: every answer must be traceable to documentation.

LLM -as- Judge

WHAT IS LLM-AS-JUDGE?

"Use a strong model (GPT-4, Claude, Gemini) to score outputs from a smaller or older model. With a clear rubric, the judge model can grade hundreds of thousands of outputs cheaply."

In plain English:

A senior engineer reviews a junior engineer's code. The senior is faster and more consistent than asking 100 humans to do it.

Best practices



Pairwise > absolute

"Which is better, A or B?" beats "rate this 1-10".



Use a clear rubric

Few-shot examples and structured JSON output reduce randomness.



Randomize order

Position bias is real — alternate which side is "A".



Multiple judges

Average across 2-3 judge models for higher reliability.

RAGAS: Four Pillars for RAG Evaluation



Faithfulness

Are all claims in the answer supported by the retrieved context?



Answer Relevance

Does the answer actually address the question that was asked?



Context Precision

Are the most relevant retrieved chunks ranked highest?



Context Recall

Did retrieval find all the information needed to answer?



Open-source

Pluggable Python library; works with any LLM and embedding model.



Use case

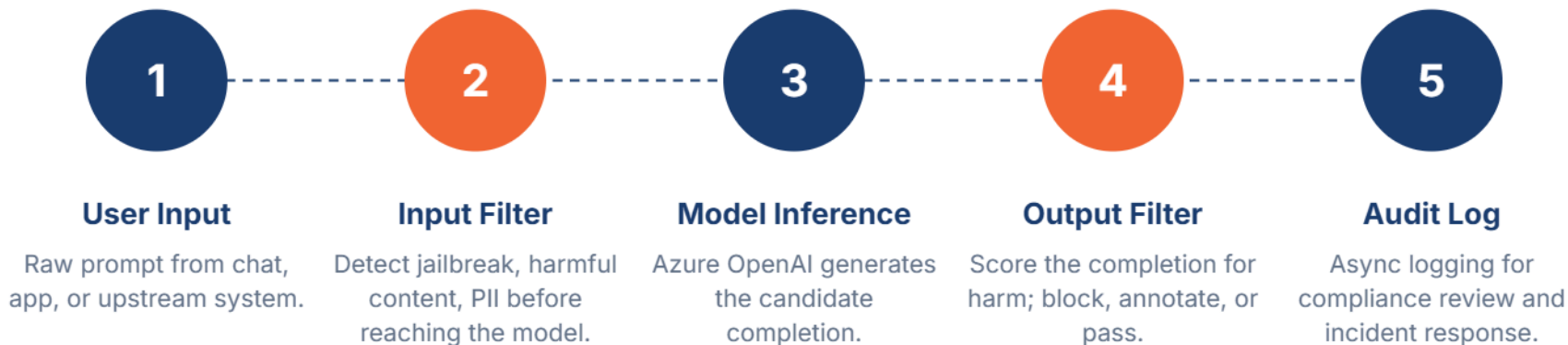
Track these four metrics on a fixed eval set — detect regressions before users do.

Azure OpenAI Safety System



Defense in Depth: Five-stage Pipeline

Every production LLM deployment should pass user input through multiple safety layers before and after the model.



No single layer is perfect — combining input, model, and output safety produces materially safer behavior than any one alone.

Azure Content Safety: What is Included



Built into deployments

Content filters configured per Azure OpenAI deployment — no extra integration code required.



Configurable per use case

Different thresholds for consumer chatbots vs internal copilots vs developer tools.



Regional deployment

Available across regions including Southeast Asia for data residency.



Annotated logging

Every flagged input/output is logged with category, severity, confidence.



API surface

Standalone Content Safety API also available for non-Azure-OpenAI workloads.



Compliance fit

Aligns with EU AI Act, Singapore MAS FEAT, and other regulatory expectations.

Four Harm Categories: Evaluated for Prompt and Completion

Hate · Sexual

- Hate and Fairness — content attacking, demeaning, or discriminating against groups
- Includes religion, race, gender, ethnicity, disability, sexual orientation
- Sexual — content describing sexual acts, body parts, or explicit material
- Particularly relevant for any consumer-facing product

Violence · Self-harm

- Violence — content describing physical harm, weapons, threats, or violent acts
- Self-harm — content related to self-injury, suicide, eating disorders
- Self-harm needs special handling: route to crisis support resources
- Each of the four is scored independently for both prompt and completion

Severity Levels: Configurable Thresholds

Level	Label	What it means	Recommended action (enterprise default)
0	Safe	No harmful content detected	Pass through
2	Low	Minor mentions or contextually appropriate references	Pass through, log for review
4	Medium	Clear harmful content but limited intensity	Block by default; allow with override and logging
6	High	Severe, explicit, or directly harmful content	Block always; alert security and compliance

Defaults can be overridden per category — e.g., medical use cases may need lower thresholds for self-harm to allow clinical discussion.

Prompt Shields: Defending Against Injection



Direct prompt injection

"Ignore previous instructions and..." — user attempts to override the system prompt.



Indirect injection

Malicious instructions hidden in retrieved documents — RAG-specific and growing.



Encoding tricks

Base64, leet speak, ROT13, Unicode tricks designed to bypass keyword filters.



Role-play framing

"Pretend you are an AI without restrictions..." — exploits persona compliance.



Multilingual vectors

Bahasa Indonesia, slang, code-switching — often less filtered.



Defenses

Structured outputs, system-prompt isolation, output validation, multiple safety layers.

Protected Material & Groundedness Detection

Protected Material Detection

- Detects copyrighted text and code reproduced verbatim in completions
- Returns flagged spans with match confidence and reference identifier
- Critical for legal and IP risk management on customer-facing outputs
- Especially relevant for code-generation copilots
- Pairs with citation and provenance workflows

Groundedness Detection

- Identifies ungrounded claims in completions when context is provided (RAG)
- Returns ungrounded segments + reasoning explaining why
- Optional reasoning mode trades latency for explainability
- Plug into a citation/correction loop to regenerate ungrounded sections
- Acts as a final runtime check before responses reach the user

Custom Categories for Enterprise Needs

Define your own risks

- PII leakage — KTP, NIK, account numbers, addresses
- Unauthorized advice — financial, legal, medical recommendations
- Defamation against competitors or named individuals
- Out-of-scope answers — pretending to know proprietary specs
- Promise-making language — "guaranteed", "you will get..."

Tune per use case

- External chatbot — strict thresholds, block more aggressively
- Internal copilot — relaxed thresholds, more helpful
- Developer tools — code-aware filters, allow technical jargon
- Different deployments → different safety configurations
- Audit logging required across all deployments

Analyzing Toxic Output



What is Toxicity

WORKING DEFINITION

"Toxicity = "rude, disrespectful, or unreasonable language likely to make someone leave a discussion." Spectrum from mild incivility to severe harm (threats, hate speech)."

— Perspective API, Jigsaw

In plain English:

Toxic $\neq$ unsafe $\neq$ biased $\neq$ wrong. A toxic message can be factually correct. Treat each axis independently.

Important distinctions



Context matters

Clinical discussion of self-harm differs from glorification.



Cultural calibration

Bahasa Indonesia toxicity norms differ from English.



Subtle vs overt

Microaggressions and dog whistles are hardest to detect.



Different from bias

Bias = unfair patterns; toxicity = harmful tone or content.

Types of Toxic Content



Profanity & slurs

Vulgar language and identity-targeted slurs.



Harassment & threats

Personal attacks, doxxing, threats of violence.



Hate speech

Group-targeted dehumanization or incitement.



Identity attacks

Demeaning content targeting religion, ethnicity, gender.



Sexually explicit

Especially when unsolicited or directed at minors.



Subtle toxicity

Microaggressions, dog whistles, coded language.

Detection Tools: Which to Choose

Tool	Coverage	Cost / hosting	Notes
Perspective API	Toxicity, severe toxicity, identity attack, insult, profanity, threat	Free at low volume (Jigsaw / Google)	Strong English; weak Bahasa Indonesia
Detoxify	Multi-label toxicity in multiple languages	Open-source, self-hosted	Tunable; good fit for on-prem deployments
OpenAI Moderation	Sexual, hate, harassment, self-harm, violence	Free with OpenAI API usage	Tightly integrated with OpenAI completions
Llama Guard 2 / 3	Configurable taxonomy via prompt	Open weights, self-hosted	LLM-based, customizable categories
Azure Content Safety	Hate, sexual, violence, self-harm + custom categories	Azure consumption pricing	Default choice when already on Azure stack

Red Teaming: Finding Harms Before Users Do



What it is

Structured adversarial testing to surface harmful behaviors before production.



Manual red teaming

Human experts probe with creative jailbreaks — high quality, low scale.



Automated red teaming

Generator LLMs produce thousands of adversarial prompts at scale.



PyRIT (Microsoft)

Python Risk Identification Toolkit — open-source framework for systematic LLM red teaming.



Garak · Giskard

Open-source vulnerability scanners that probe known LLM weaknesses.



Cadence

Quarterly red team engagements with attack categories tied to business risk.

Adversarial Testing & Prompt Injection



Direct injection

"Ignore previous instructions" — most common, easiest to detect with shields.



Indirect injection

Malicious instructions hidden in retrieved documents — harder to detect.



Encoding tricks

Base64, leet speak, ROT13, Unicode designed to bypass keyword filters.



Role-play framing

"Pretend you are an AI without rules" — exploits persona compliance.



Multilingual vectors

Bahasa, code-switching, regional slang — often less filtered.



Defenses

Structured outputs, system-prompt isolation, output validation, multiple layers.

Monitoring & Evaluation Pipelines



Logging

Capture input, output, safety scores per category, model version, timestamp.



Sampling strategy

Risk-weighted: review 100% of high-severity flags, % of medium flags.



Human review

Trained reviewers triage flags; feedback loops into model and filter tuning.



KPIs

Toxicity rate, false-positive rate, time-to-detect, repeat-offender users.



Incident runbook

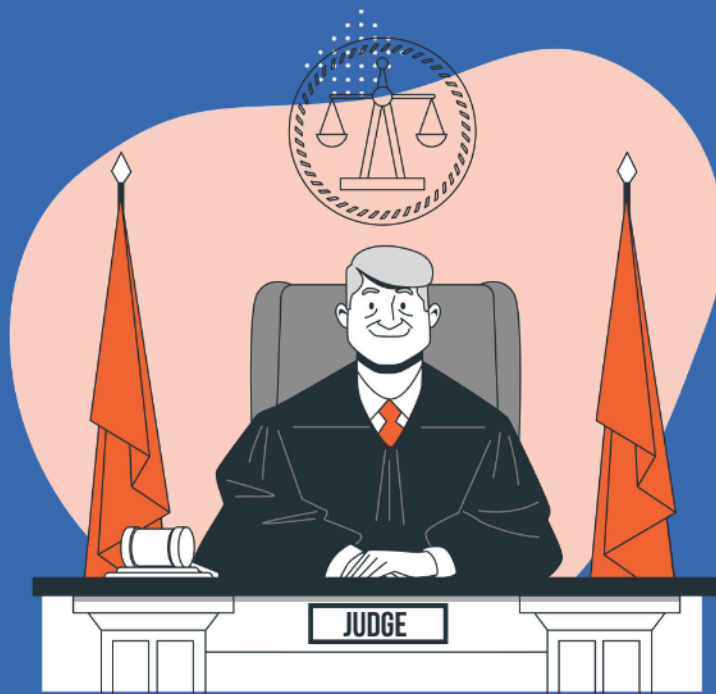
Severity levels, on-call rotation, comms templates, post-mortem process.



Compliance

Reports flow to risk committee and form part of regulatory examinations.

Bias Mitigation Techniques



Sources of Bias in LLMs



Training data bias

Web corpora skew Western, English, and toward those with internet access.



Annotation bias

Human labelers bring values; instruction-tuning encodes them into the model.



Algorithmic bias

Loss functions, sampling, architecture choices amplify some patterns.



Deployment bias

Use beyond design intent — English-trained model for Bahasa queries.



Feedback loops

Production outputs become future training data, reinforcing existing biases.



Evaluation bias

Benchmarks themselves can be biased; high benchmark scores $\neq$ low real-world bias.

Types of Bias to Look For



Demographic

Gender, race, religion, age, disability — differential treatment.



Geographic / cultural

Western defaults, urban vs rural, country-of-origin assumptions.



Linguistic

Bias against Bahasa Indonesia, Javanese, Sundanese; favoritism toward English.



Socioeconomic

Assumptions about wealth, education, occupation embedded in completions.



Confirmation bias

Models reinforce user assumptions instead of challenging them.



Recency bias

Heavy weight on recent training data — older context underrepresented.

Pre-training & Fine-tuning Mitigation



Pre-training (upstream)

- Data curation — filter low-quality, toxic, biased sources before training
- Balanced corpora — active sampling across demographics and languages
- Deduplication — prevent overweighting common phrasings
- Datasheets and Data Cards documenting sources and limitations
- Mostly upstream for enterprise teams using pre-trained models



Fine-tuning & alignment

- Supervised fine-tuning on curated, balanced datasets
- RLHF — Reinforcement Learning from Human Feedback
- DPO — Direct Preference Optimization (simpler, more stable)
- Constitutional AI / RLAIIF — feedback against an explicit principles document
- Trade-off: alignment tax — measure capability loss explicitly

Inference-time Mitigation: What you Can do Today



System-prompt guardrails

Explicit instructions about respectful, balanced, culturally aware behavior.



Output rewriting

Post-process completions through a second model trained for fairness corrections.



Counterfactual augmentation

Inject diverse examples in retrieval and few-shot prompts.



Persona controls

Define and enforce explicit persona policies; reject persona-shifting inputs.



Re-ranking

Sample multiple completions; re-rank by safety, fairness, groundedness scores.



Why this matters

Inference-time controls are the only lever most enterprise teams fully control.

Bias Measurement Frameworks

Framework	What it measures	Format	Best for
StereoSet	Stereotypical bias across gender, race, religion, profession	Cloze + sentence pairs	Comparing models on stereotype tendency
BBQ	Bias in QA across nine demographic dimensions	Multiple-choice QA	QA-focused bias evaluation
BOLD	Bias in open-ended generation (sentiment, regard)	Prompted completions	Long-form generation
WEAT / SEAT	Implicit association in word/sentence embeddings	Embedding-based test	Encoder-level bias diagnosis
HELM	Holistic Evaluation of Language Models — broad benchmark suite	Multi-task	Comprehensive comparative evaluation

Governance & Best Practices



AI ethics committee

Standing review board with cross-functional representation.



Regulatory mapping

Document compliance with EU AI Act, MAS FEAT, ASEAN AI Guide.



Model cards

Public-facing documentation: intended use, limitations, evaluation results.



Continuous monitoring

Bias does not stay fixed — re-evaluate after every model or data update.



MLOps integration

Bias checks as automated gates — block deployment if regression exceeds threshold.

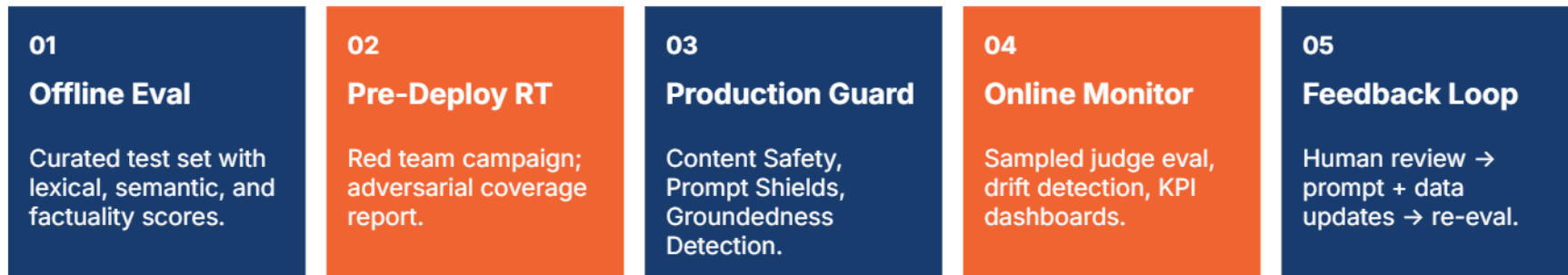


External assurance

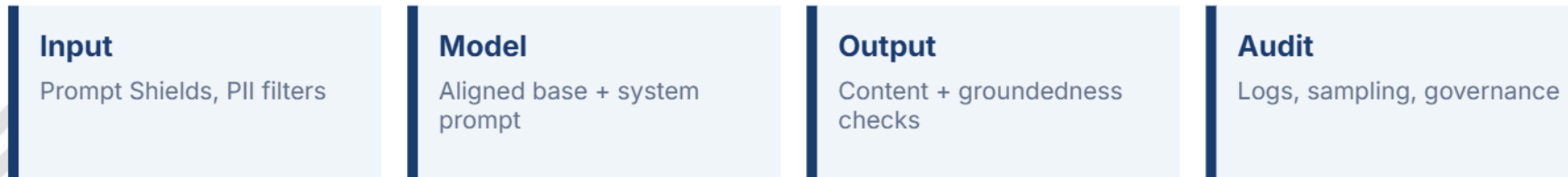
Independent audit and red teaming for high-stakes systems builds trust.

Putting it all together: End-to-end Framework

A production-grade LLM evaluation and safety system layers offline, runtime, and feedback components.



LAYERED DEFENSES



Key Takeaways



No single metric

Combine lexical, semantic, factuality, and safety metrics. Each catches a different failure mode.



Continuous, not one-time

Safety and bias evaluation must run continuously — every model update introduces new risk.



Defense in depth

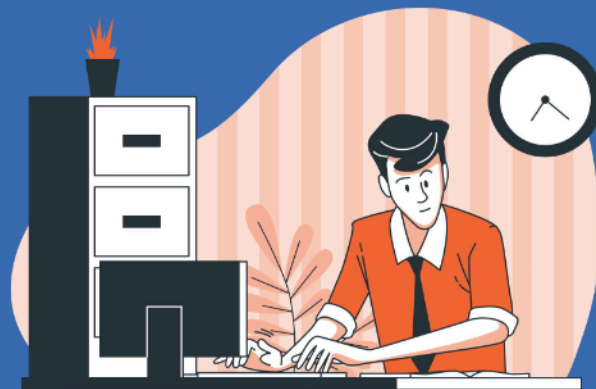
Azure OpenAI gives a strong baseline; customize per use case across input, model, and output.



People + tools

Trustworthy LLM systems require governance, tooling, and people working together.

Guided Hands-on



What You Will Walk Away With



Run all three lexical metrics

BLEU, ROUGE, and METEOR on real Mitsubishi customer-support Q&A pairs.



Compare meaning, not just words

Use embeddings + BERTScore to catch what BLEU and ROUGE miss.



Use an LLM-as-Judge

Score outputs with a structured rubric and JSON-mode prompting.



Wire in safety filtering

Send test prompts through a content moderation API and read the category scores.



Build a unified eval report

Combine every signal into a single ship/review/block decision.

The Five Stages at a Glance

Each stage takes about 15–20 minutes. Watch for the 💡 Aha-moment notes — that is when to pause and discuss.

STAGE 1



Lexical

BLEU + ROUGE +
METEOR on 6
Mitsubishi Q&A pairs.

STAGE 2



Semantic

Cosine similarity +
BERTScore. Compare
to lexical.

STAGE 3



Judge

GPT-4o-mini with a
1–5 rubric in JSON
mode.

STAGE 4



Safety

OpenAI Moderation
API on 5 test prompts.

STAGE 5



Combine

Heatmap dashboard +
decision rule.

Total time: ~90 minutes. The instructor walks through every cell live; participants follow along.

Before we start: Set up Colab Secrets

No API keys are hardcoded anywhere in the notebook. Set up the Colab Secrets panel before the session starts.

Required

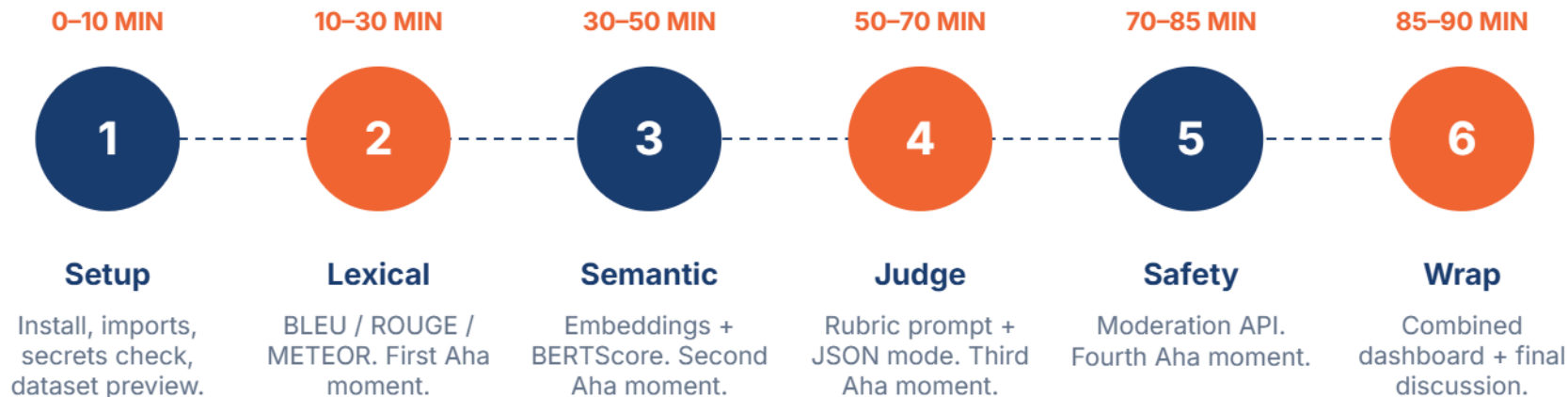
- AZURE_OPENAI_API_KEY — if you have an Azure OpenAI resource
- AZURE_OPENAI_ENDPOINT — your Azure endpoint URL
- Notebook shows the Azure pattern in commented code
- Skip these if using OpenAI directly — that is the default path

Optional

- OPENAI_API_KEY — used by Stage 3 (judge) and Stage 4 (safety)
- Add via the  Secrets icon in the Colab left sidebar
- Enable "Notebook access" so the notebook can read the value
- Each participant uses their own personal key

Time Allocation Across the 90 Minutes

A loose schedule — adjust based on the room. Save buffer for discussion at every Aha moment.



LLM Evaluation, Safety & Toxicity Detection

Learning Goals

By the end of this session, participants will be able to:

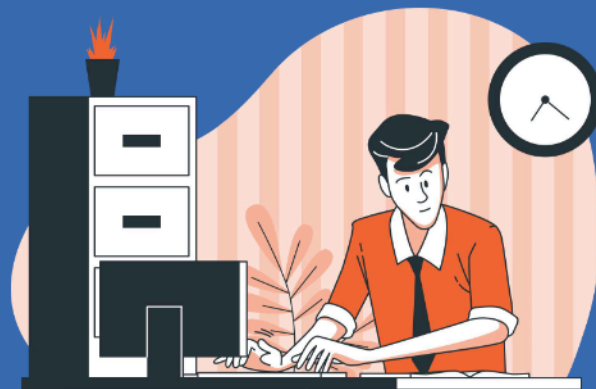
- Run all three lexical metrics
- Compare meaning, not just words
- Use an LLM-as-Judge
- Wire in safety filtering
- Build a unified eval report

Setup

Participants can use:

- Google Colaboratory:
https://drive.google.com/file/d/12YTdg9V3NVPhkjlw4qzp_ulcvTGhODoZ6/view?usp=sharing

Assignment



Why this Assignment Exists



Cement the lecture

Lectures fade fast. Building one real evaluation makes the metrics stick for years.



Build a portfolio piece

Both options produce a notebook + report you can show in interviews and reviews.



Practise communicating with non-engineers

The 3–5 page report is the hardest part — that is the point. Engineers are paid to explain.



Develop an evaluation mindset

After this assignment you will instinctively ask "how would I measure this?" before shipping.

Two Options: Pick the One That Excites You

You only complete one. Both options are graded on the same rubric.



Option A — The Three-Metric Showdown

- Run BLEU, ROUGE, METEOR, BERTScore, and cosine similarity on a real dataset
- Use 20–50 examples — the starter Mitsubishi set or a Hugging Face dataset
- Find 5 cases where the metrics disagree and explain why
- Recommend one metric (or combination) for production gating
- Pick this if you enjoyed Stages 1 and 2 of the hands-on



Option B — Red Team a Chatbot

- Design 20 adversarial prompts across 4+ attack categories
- Run them against an in-notebook target chatbot we provide
- Categorize each result as refused / partial / succeeded
- Propose 3 concrete defenses and identify residual risk
- Pick this if you enjoyed Stage 4 and like adversarial thinking

Option A: What Done Looks Like



Step 1 — Choose a dataset

20–50 (reference, candidate) pairs. Starter Mitsubishi set OR a Hugging Face dataset.



Step 2 — Compute all 5 metrics

BLEU, ROUGE-L, METEOR, BERTScore, cosine similarity. Build a comparison DataFrame.



Step 3 — Make 2 visualizations

Anything that helps a non-technical reader see where the metrics agree or disagree.



Step 4 — Document 5 failure cases

Pairs where metrics sharply disagree. Explain the linguistic phenomenon causing each gap.



Step 5 — Recommendation memo

200–400 words to a non-technical reader: which metric, why, and the biggest risk.

Option B: What Done Looks Like



Step 1 — 20 adversarial prompts

Cover at least 4 of 6 attack categories (direct injection, role-play, encoding, multilingual, etc.).



Step 2 — Run all prompts

Send each through the target chatbot. Capture and store the responses.



Step 3 — Score each result

Refused / Partial / Succeeded. One sentence reasoning per row.



Step 4 — Aggregate and visualize

Success rate per category. A stacked bar chart works well.



Step 5 — Remediation memo

200–400 words. Three concrete defenses + residual risk after them.

Deliverables: Submit All Three



The notebook (.ipynb)

Every cell executed, all output visible, no leftover # TODO comments. You can follow example here: [Session 38 Assignment_LLM Evaluation, Safety & Toxicity Detection.ipynb](#)



The report (3–5 pages, PDF)

Written for a non-technical reader. Cover what you did, what you found, what you recommend.



The presentation (5 slides)

Clean summary suitable for the next session. You will present briefly to peers.

Ground Rules: Read Carefully



No hardcoded API keys, ever

All secrets must come from Colab Secrets. Notebooks with hardcoded keys lose 20 points.



Sample data must use Mitsubishi only

No competitor brands in any sample data, prompts, or examples.



Original work

Code from the hands-on notebook is fair game. Borrowing whole solutions from peers is not.



Help is available

Consult your instructor for a better understanding of the assignment.



Thank You